

FINAL PROJECT REPORT

LIVEWELL Health Assessment & Consultation Booking

Group name: Belavatha KrishnegowdaNTuT

Members : Tuyet Tu and Neha Belavatha Krishnegowda

1. OUR INTEREST

We are interested in this project and data domain as both of us are personally committed to the areas of health and wellness. We find this subject particularly significant since one of our members has been going through her weight-loss and fitness process on her own. In the process, she would not know many times whether she was doing the right thing and had to experiment with trying things out. That experience demonstrated to her that it is quite difficult to find the right guidance that is clear, trustworthy, and, in fact, individualistic.

We have a particular interest in health analytics, personalized recommendations of wellness, patient-centered technology, and how data is used to facilitate improved lifestyle and treatment choices. As we both are in Data Science domain, we want to challenge ourselves into exploring about data driven solutions to healthcare issues

Yet another reason why this project is interesting to us is that nowadays most individuals rely on Google search or random websites searching for health advice, and the content they get is either too general, unreliable or lacks personalization. Such a system can unite all the relevant information, assessments and recommendations to be in one location and hence allow the users to have an easier access to more structured and meaningful guidance.

We consider this project to be relevant to us as it relates a personal experience to one of the fields of technology that we are truly interested in learning more about.

2. BACKGROUND

Obesity has become one of the most significant public health concerns in the United States. A new study published in the Journal of the American Medical Association (JAMA), projecting that

by 2035, nearly half of the American adult population - around 126 million people - will become obese (Johnson et al., 2026). Despite the severity of the issue, appropriate medical intervention remains limited. There are several barriers for care services namely lack of access to primary care services, high cost and also psychological barriers. A survey conducted by PatientPoint reveals that nearly half of Americans avoid a doctor's visit for top reasons such as concerns about affording care (52%) or anxiety about potential procedures or tests (40%) (PatientPoint, 2023).

3. SOLUTION OBJECTIVES

LiveWell is a health consultancy provider addressing this unmet need by giving personal health care guidance at an affordable price through a digital health platform. By integrating machine learning-based obesity classification models, LiveWell empowers our customers to obtain an instant, data-driven preliminary assessment of their obesity risk before making any commitment to a consultation booking with our specialists.

4. DATA DESCRIPTION

Livewell allows users to register a personal profile that comprises following attributes: email, phone number, full name (first name and last name), gender, date of birth, height, weight. The system also tracks age and BMI for each user that can be derived from the user's date of birth, height, and weight respectively. Each user's email is unique and serves as their primary identifier. Each user's phone is also unique across all users. A new user registration requires that neither the provided email nor phone number is already associated with an existing account. (For POC purposes, the registration process is demonstrated through a form-based input interface with constraint validation; a full authentication pipeline including sign-in and log-in functionality is outside the scope of this project implementation).

Once registered, a user completes 0 to many assessment sessions through a structured evaluation questionnaire. Each assessment session belongs to 1 and only 1 user. Each time a user completes an assessment, the system records the date and time at which the session starts. The combination of the user's email and the session start datetime uniquely identifies an assessment session.

The questionnaire is designed with each question storing its unique question text, question type, a required flag indicating whether the question must be answered before the assessment can be submitted, and an active flag indicating whether the question is currently presented to users. The active flag allows questions to be retired from future assessments without losing historical response data. Each question is one of three types: boolean for yes/no questions, single choice for questions with one selectable option, and multiple choice for questions where more than one option may be selected.

A question has one or many predefined question options. Each question option belongs to 1 and only 1 question. Each question option stores a display text shown to the user, a machine-readable value passed to the ML model, and a display order controlling the sequence in which options are presented to the user. The combination of the question text and the option display text uniquely identifies a question option. This design allows new questions or question options to be introduced without structural changes to the database.

A session records the user's selected question options across all active required questions. Specifically, each response within a session is identified by the combination of the user's email, the session start datetime, the question text, and the selected question's option text — capturing exactly which option the user chose for each question in that session. For single choice and boolean questions, the user selects exactly 1 option per question; for multiple choice questions, the user selects 1 to many options. There is no open-ended question type.

The current questionnaire covers 12 lifestyle factors – all are required questions. The system to introduce additional optional, non-required questions in the future. The required questions needed for obesity prediction are:

- Whether the user has a family history of being overweight
- Whether the user frequently consumes high-calorie foods
- A frequency of vegetable consumption with a scale from 1 to 3
- The number of main meals per day
- A frequency of consuming food between meals that can be selected from never, sometimes, frequently, always
- Daily water intake (scale from 1 to 3)
- Whether the user monitors their calorie intake
- Physical activity frequency (scale from 0 to 3)
- Time spent using technology (scale from 0 to 3)
- Frequency of alcohol consumption (Never, Sometimes, Frequently, Always),

- Whether the user smokes
- Main mode of transportation (Automobile, Bike, Motorbike, Public Transportation, Walking)

Once all required questions are answered, the lifestyle inputs are combined with the user's biometric data — specifically age, gender, height, and weight — and passed to a pre-trained machine learning classification model. The model produces a predicted obesity classification stored as part of the assessment result, across one of 7 levels: Insufficient Weight, Normal Weight, Overweight Level I, Overweight Level II, Obesity Type I, Obesity Type II, and Obesity Type III.

Each assessment session stores at most one predicted obesity level and its associated computed nutrition values — daily caloric intake, protein, carbohydrates, and fat in grams — directly within the session record. These fields are populated when the user requests results after completing the questionnaire, and remain null for in-progress sessions.

LiveWell maintains a predefined set of recommendation rules, each uniquely identified by its obesity level. Each rule carries a description summarizing the health guidance for that obesity level, and a tailored message presented directly to the user to explain their result and encourage next steps. Each rule also stores four formula parameters used to compute a personalized nutrition plan: a calorie adjustment factor applied to the user's Total Daily Energy Expenditure (TDEE) to determine the target caloric intake — a positive value indicates a caloric surplus for weight gain while a negative value represents a deficit for weight loss; the protein per kilogram parameter defining the recommended daily protein intake relative to the user's body weight; and the carbohydrate ratio and fat ratio defining the proportion of remaining calories to be allocated to carbohydrates and fat respectively.

Each completed assessment session is associated with 1 and only 1 recommendation rule, determined by its predicted obesity level. A recommendation rule may be associated with 0 to many completed assessment sessions. Upon completion, the system retrieves the matching recommendation rule and uses its formula parameters to compute the personalized nutrition values, which are stored directly within the session record.

A user can revisit their assessment history to review past results. Completed assessments may be deleted by the user. For sessions that were started but not completed, the user may return to finish the questionnaire or retrieve results if responses were previously saved.

LiveWell maintains a catalog of consultants. Each consultant's email is unique and serves as their primary identifier. Each consultant's phone number is also unique across all consultants. For each consultant, the system also tracks a first name, last name, and a description.

A consultant is associated with 1 to many specializations. A specialization is associated with 1 to many consultants. The specialization values are: Weight Management, Clinical Nutrition, Sports Nutrition, Holistic Nutrition, and Pediatric Nutrition.

A consultant has 0 to many availability records. Each availability record belongs to 1 and only 1 consultant. Each availability record defines a date and time the consultant is available for booking, along with a booking status — initially open when created, and updated to booked once a user reserves the slot. An availability record is uniquely identified by the combination of the consultant's email and the slot date and time.

A user places 0 to many bookings. Each booking belongs to 1 and only 1 user. Each booking is associated with 1 and only 1 availability record, and each availability record is associated with at most 1 booking. A user cannot place more than one booking for the same date and time slot across different consultants. A booking is uniquely identified by the combination of the user's email and the availability record it is associated with — that is, by the combination of the user's email, the consultant's email, and the slot date and time.

A user can view their booking history and manage 0 to many of their existing bookings. The user may cancel a booking, upon which the slot is released and becomes available for other users. The user may also reschedule a booking by selecting a new consultant or time slot, upon which the original slot is released and the new slot is reserved.

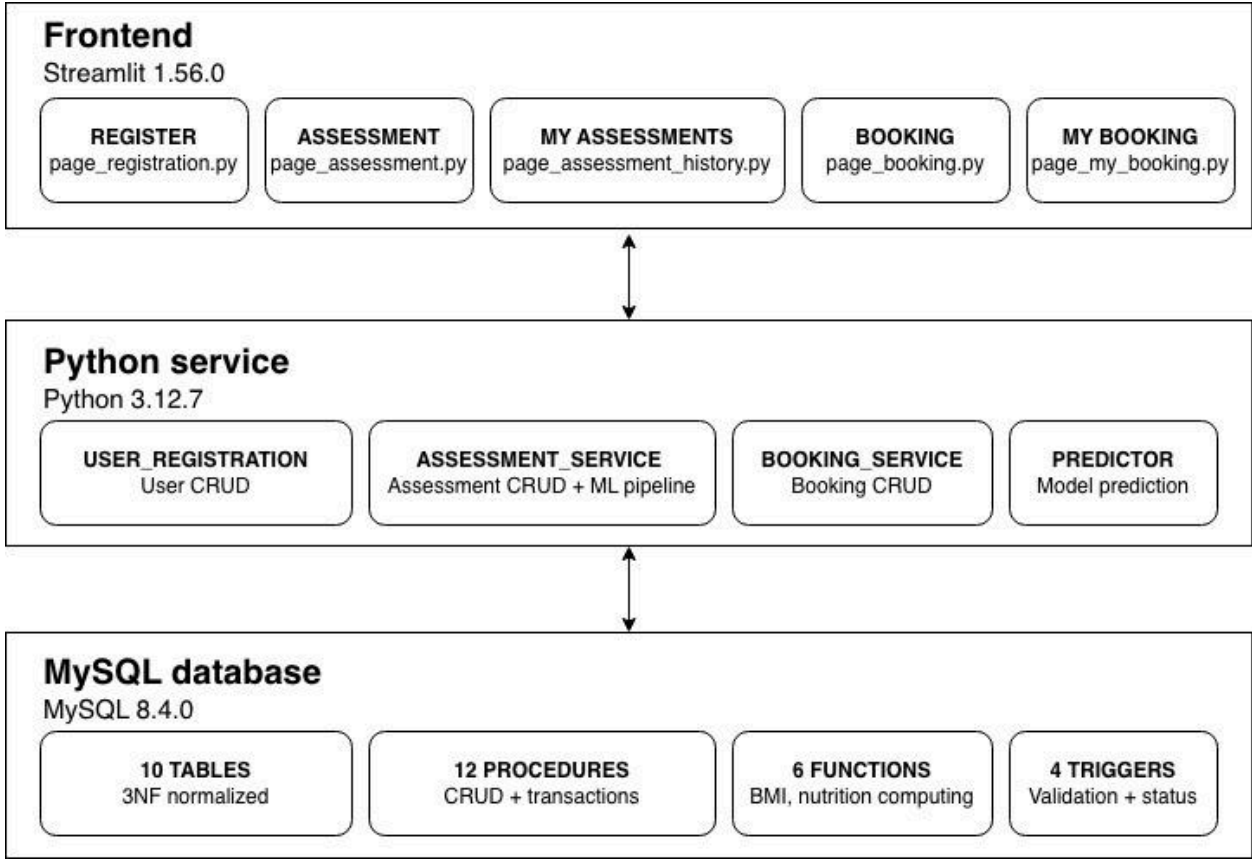
5. SYSTEM DESIGN

5.1 Technological Stack choice

Component	Technology
Frontend	Streamlit 1.56.0
Backend	Python 3.12.7

Database	MySQL 8.4.0
ML Libraries	Scikit-learn, Pandas, Numpy, Joblib
ML Model	LightGBM 4.6.0
DB Connector	Mysql-connector-python 9.6.0
Hosting	Localhost (for POC)

5.2 System Architecture



The system follows a three-layer architecture consisting of a Streamlit frontend and a Python service layer directly connected to a MySQL database.

The frontend comprises five modules: Registration, Assessment, My Assessments, Booking, and My Bookings. Each module calls Python service functions which handle all business logic including input validation, ML inference, recommendation rule matching, and consultant filtering. The pre-trained LightGBM obesity classification model is loaded at application startup and called directly within the assessment service to generate predictions.

The database stores all persistent data. It uses stored procedures for complex multi-step operations, user-defined functions to compute nutrition values, and triggers to manage booking status and validate data automatically.

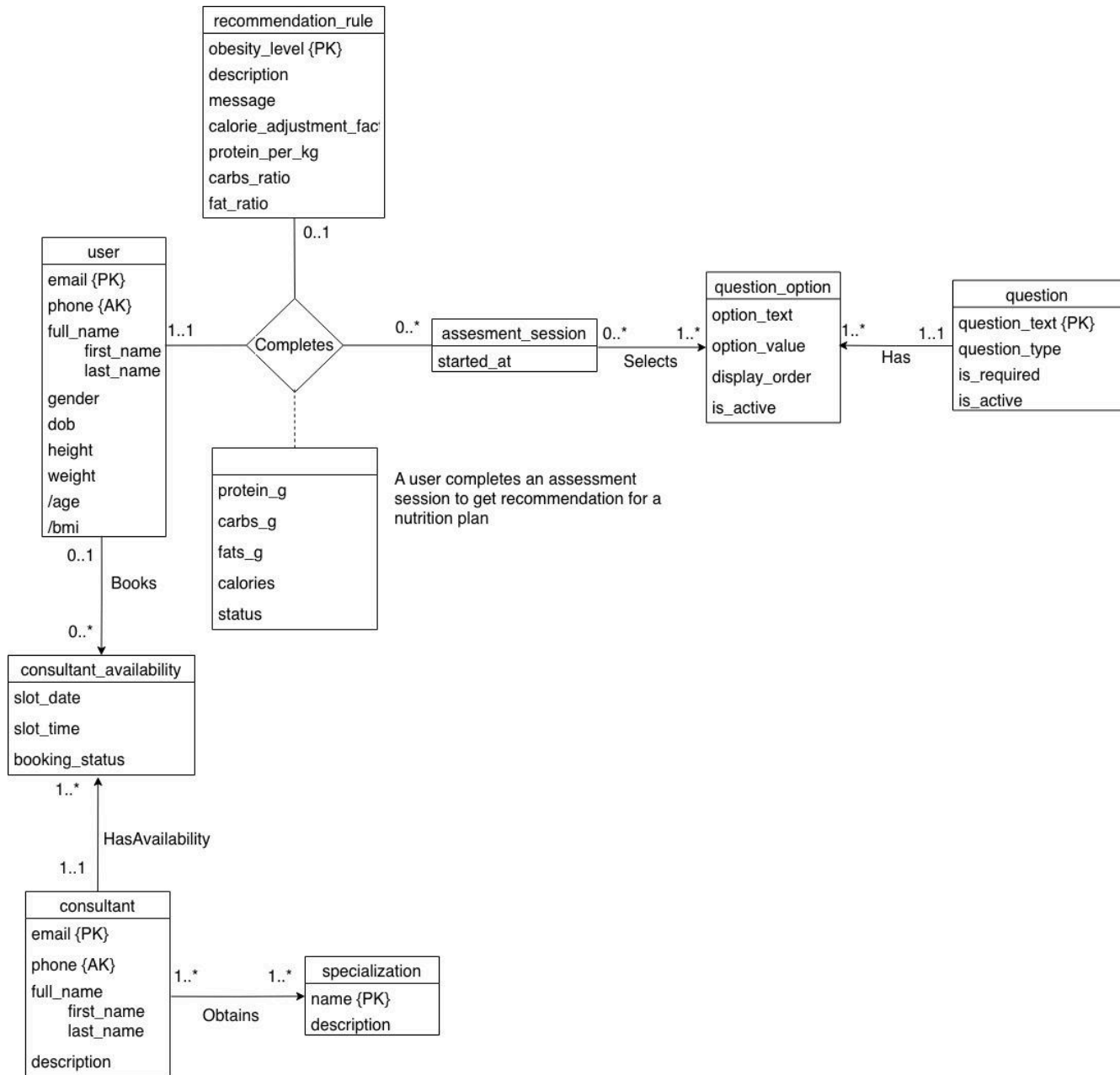
Note: Since no authentication is implemented for POC, user identity is resolved by email input at the point of each form submission.

Machine Restrictions

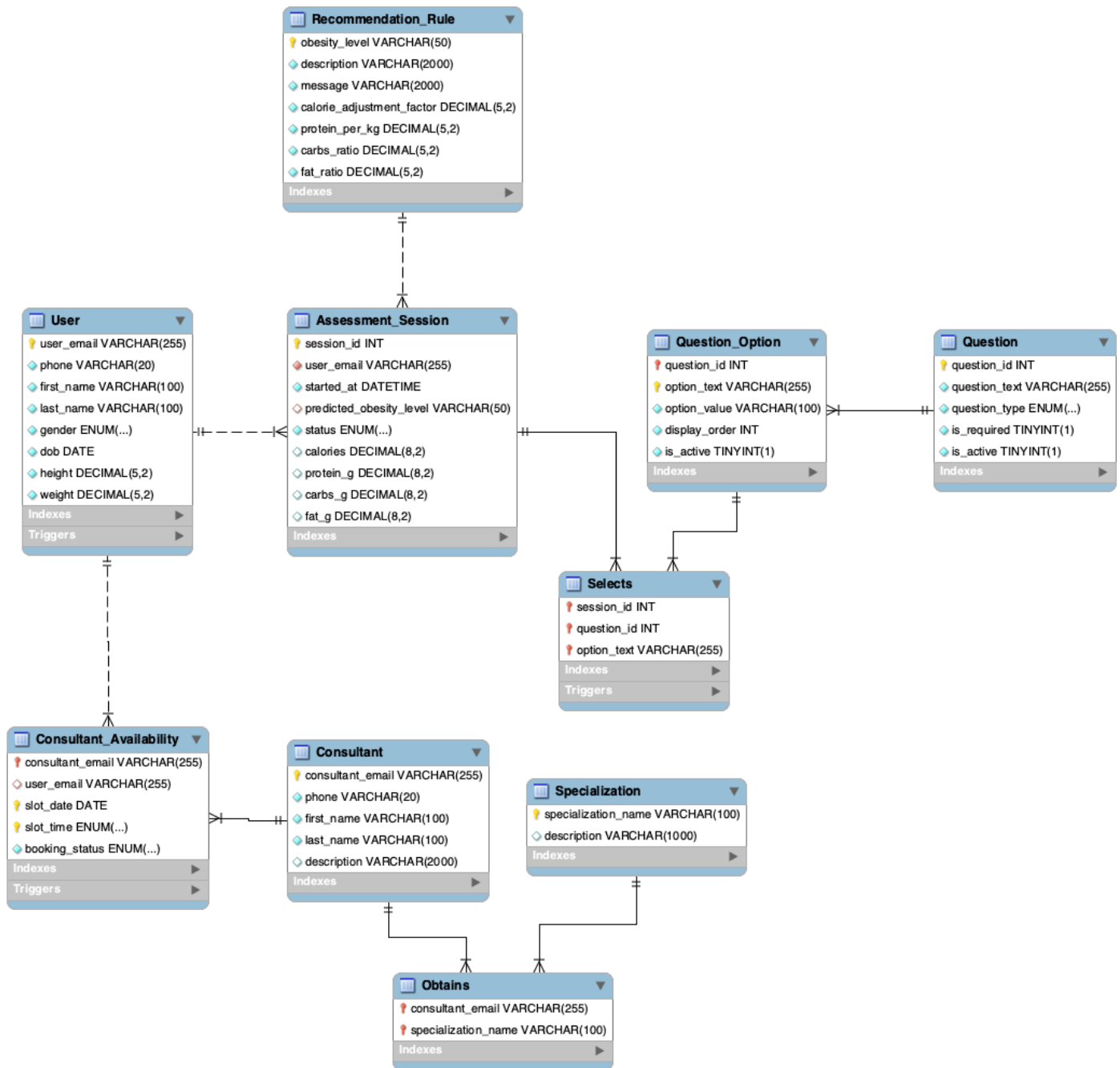
1. No specialized hardware is required.
2. The application runs on any standard machine with Python 3.12.7 and MySQL 8.4.0 installed.
3. All components run locally on a single machine for POC purposes.
4. No GPU is needed as the ML model runs on tabular data with minimal compute cost.

6. DATABASE DESIGN

6.1 Data conceptual design



6.2 Data Logical design



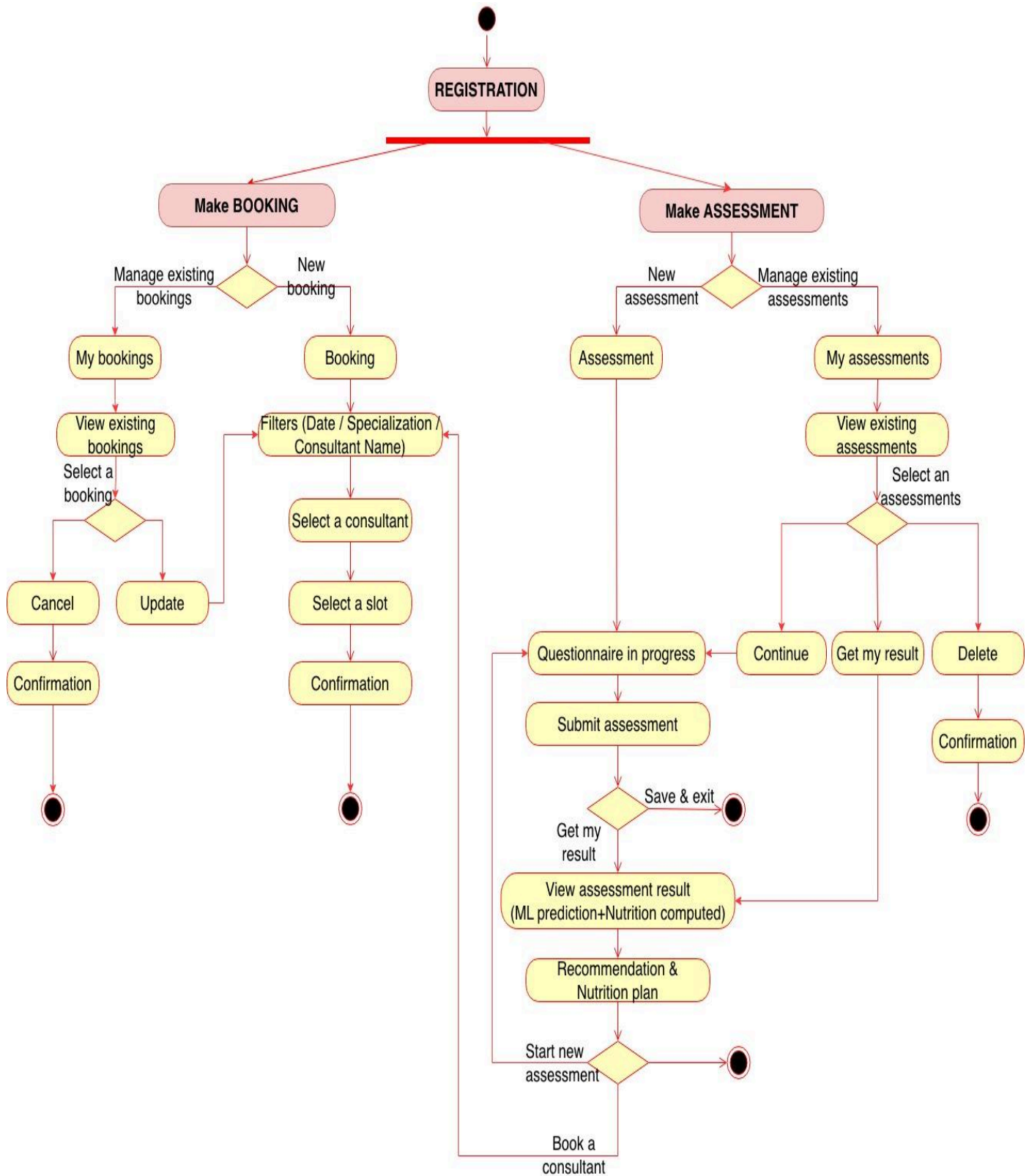
Surrogate Keys Design Decision

Two surrogate keys were introduced in the logical design, originally identified natural keys:

`question_id` — The conceptual design identifies `question_text` as the unique identifier for a question. In the initial implementation, `question_text` was defined as `VARCHAR(1000)`, which exceeded MySQL's maximum index key length of 3072 bytes under `utf8mb4` encoding, making it impossible to set a `UNIQUE` constraint. Although the column was later reduced to `VARCHAR(255)` to resolve this, `question_id` was retained as the primary key for two practical reasons: it serves as a more efficient foreign key in the `Selects` table, and it simplifies the `submit_response` procedure which takes `question_id` as a parameter rather than passing the full question text.

`session_id` — The conceptual design identifies the combination of `user_email` and `started_at` as the natural composite key for an assessment session. While semantically correct, using this composite key as a foreign key in the `Selects` table and as an `OUT` parameter in the `start_assessment_session()` procedure would require passing two values across every backend operation. A surrogate key `session_id INT AUTO_INCREMENT` reduces this to a single integer reference, simplifying all service function calls, procedure parameters, and Streamlit session state management.

7. USER FLOW



The activity diagram illustrates the main user flow of the LiveWell system. The process begins when the user enters the system and completes **REGISTRATION**. After registration, the workflow splits into two main activities that the user can perform: making **ASSESSMENT** and making **BOOKING**.

On the ASSESSMENT flow, a user can decide to create a new assessment or manage his/her existing assessments.

- For a new assessment, the user has to complete a health questionnaire. Upon submitting the questionnaire, the user's responses are saved and the user is prompted to either get results immediately or save and exit.
 - If the user chooses to get results, the system runs a machine learning model to predict the user's obesity level, computes a personalized nutrition plan, and displays the result alongside a tailored recommendation message. After viewing the result, the user can decide to make a booking, start a new assessment, or end the process.
 - If the user chooses to save and exit, the session is saved as in-progress and ends the process.
- For existing assessments, the user views and selects an existing assessment from the list:
 - For a completed assessment, the user can delete it. After confirmation, the workflow reaches the end state.
 - For an incomplete assessment without completed responses, the user can continue to fill in the questionnaire. The flow then follows the new assessment procedure from the questionnaire step.
 - For an incomplete assessment without completed responses, the user can get the results. The flow then follows the get results procedure above. After viewing the result, the user can decide to make a booking, start a new assessment, or end the process.

On the BOOKING flow, a user can decide to create a new booking or manage his/her existing bookings.

- For a new booking, the user has options to search for available consultants by filtering by date, specialization or consultant names. After selecting a suitable consultant, the

user selects an available date and time slot. The process then moves to confirmation, where the booking is finalized and the slot status is marked as booked by a trigger. After confirmation, the workflow reaches the end state.

- For existing bookings, the user views and selects an existing booking from the list to either cancel or update it.
 - If the user chooses cancellation, the process proceeds toward confirmation, where the booking is deleted and the slot marked as open by a trigger. After confirmation, the workflow reaches the end state.
 - If the user chooses updating, the process follows a new booking procedure. After a new consultant and slot is selected, the process moves to confirmation, where the old slot is released to 'open' status and the new slot is reserved to 'booked'. After confirmation, the workflow reaches the end state.

8. SYSTEM OPERATION

8.1 Stored Procedures

	Name	Description	Constraints & Validation	Error Handling
1	register_user(user_email, phone, first_name, last_name, gender, dob, height, weight)	Inserts a new user after validating uniqueness of email and phone	email and phone must not already exist in User	SIGNAL 45000: 'Email already exists'; SIGNAL 45000: 'Phone already exists'
2	get_user_by_email(email)	Retrieves user profile by email including derived age and BMI	User must exist	SIGNAL 45000: 'User not found'
3	start_assessment_session(email, OUT session_id)	Creates a new session or resumes existing	User with given email must exist	SIGNAL 45000: 'User not found'

		In-progress session for the user		
4	submit_response(session_id, question_id, option_text)	Records a user's selected option for a question within a session	session_id must exist; option must belong to an active question; duplicate not allowed for boolean/single_choice	SIGNAL 45000: 'Session not found'; SIGNAL 45000: 'Option not found or belongs to inactive question'
5	submit_assessment(session_id, predicted_obesity_level)	Finalizes session, saves prediction and computes nutrition plan using UDFs	session_id must exist and not already have a result; predicted_obesity_level must exist in Recommendation_Rule; all active required questions must have at least 1 response	SIGNAL 45000: 'Session not found'; SIGNAL 45000: 'Assessment result already exists'; SIGNAL 45000: 'Unknown obesity level'; SIGNAL 45000: 'Not all required questions have been answered'
6	search_consultants_by_specialization(specialization_name)	Returns consultants filtered by specialization name	specialization_name must exist in Specialization	SIGNAL 45000: 'Specialization not found'
7	get_available_slots(consultant_email)	Returns all open availability slots for a given consultant from today onwards	Consultant must exist; filters booking_status = 'open' and slot_date >= today	SIGNAL 45000: 'Consultant not found'

8	book_slot(user_email, consultant_email, slot_date, slot_time)	Books an available slot for a user with race condition protection via transaction and row lock	Slot must exist and be open; user must not already have a booking at the same date and time; user must not already have a booking with the same consultant on the same date	SIGNAL 45000: 'Slot not found'; SIGNAL 45000: 'Slot is no longer available'; SIGNAL 45000: 'User already has a booking at this date and time'; SIGNAL 45000: 'You already have a booking with this consultant on this date'; SIGNAL 45000: 'Consultant not found'; SIGNAL 45000: 'User not found'
9	update_booking(user_email, old_consultant_email, old_slot_date, old_slot_time, new_consultant_email, new_slot_date, new_slot_time)	Moves a user's existing booking to a new slot or consultant	Old slot must exist and be booked by this user; new slot must exist and be open; user must not already have another booking at the new date and time	SIGNAL 45000: 'Original booking not found or does not belong to this user'; SIGNAL 45000: 'New slot not found'; SIGNAL 45000: 'New slot is not available'; SIGNAL 45000: 'User already has a booking at the new date and time'
10	cancel_booking(user_email, consultant_email, slot_date, slot_time)	Cancels a booking by clearing user_email — trigger auto-resets booking_status to open	Slot must exist and be currently booked by the user	SIGNAL 45000: 'Booking not found or does not belong to this user'

11	get_booking_history(user_email)	Returns all slots booked by a user including consultant name and specializations	User must exist	SIGNAL 45000: 'User not found'
12	delete_assessment_session(session_id, email)	Deletes an assessment session and cascades to 'Selects' table	session_id must exist and belong to the user with given email	SIGNAL 45000: 'Session not found'; SIGNAL 45000: 'Session does not belong to this user'

8.2 Triggers

Name	Timing	Event	Table	Description	Logic
t_before_insert_user	BEFORE	INSERT	User	Validates that date of birth is before today	IF NEW.dob >= CURDATE() THEN SIGNAL 45000: 'Date of birth must be before today'
t_before_insert_selects	BEFORE	INSERT	Selects	Validates that the question option belongs to an active question; enforces single option constraint for boolean and single_choice questions (multiple_choice and open questions not yet deployed in this POC)	IF option not found THEN SIGNAL 45000: 'Option not found for this question'; IF question is inactive THEN SIGNAL 45000: 'Question is inactive'; IF option is inactive THEN SIGNAL 45000: 'Selected option is inactive'; IF question is single_choice or boolean AND response already exists for that question in session THEN SIGNAL 45000: 'Only one option allowed for this question type'

t_sync_booking_status_on_insert	BEFORE	INSERT	Consultant_Availability	Auto-sets booking_status based on user_email when a slot is inserted	IF NEW.user_email IS NOT NULL THEN SET booking_status = 'booked' ELSE SET booking_status = 'open'
t_sync_booking_status_on_update	BEFORE	UPDATE	Consultant_Availability	Auto-syncs booking_status when user_email is set or cleared — replaces AFTER triggers to avoid infinite loop	IF NEW.user_email IS NOT NULL THEN SET booking_status = 'booked' ELSE SET booking_status = 'open'

8.3 Functions

Name	Description	Formula / Logic
calculate_bmi(height DECIMAL, weight DECIMAL)	Calculates BMI from height (cm) and weight (kg)	$\text{weight} / (\text{height}/100)^2$
calculate_age(dob DATE)	Calculates age in years from date of birth	$\text{TIMESTAMPDIFF}(\text{YEAR}, \text{dob}, \text{CURDATE}())$
calculate_calories(weight DECIMAL, height DECIMAL, dob DATE, gender VARCHAR, calorie_adjustment_factor DECIMAL)	Calculates target daily caloric intake using Mifflin-St Jeor TDEE formula adjusted by recommendation rule factor	For Male: $\text{TDEE} = 10 * \text{weight} + 6.25 * \text{height} - 5 * \text{age} + 5$ For Female: $\text{TDEE} = 10 * \text{weight} + 6.25 * \text{height} - 5 * \text{age} - 161$ For Others: $\text{TDEE} = 10 * \text{weight} + 6.25 * \text{height} - 5 * \text{age} - 78$ Target = $\text{TDEE} * (1 + \text{calorie_adjustment_factor})$
calculate_protein(weight DECIMAL, protein_per_kg DECIMAL)	Calculates daily protein in grams	$\text{weight} * \text{protein_per_kg}$
calculate_fat(calories DECIMAL, protein_g DECIMAL, fat_ratio DECIMAL)	Calculates fat grams from remaining calories after protein	$\text{remaining} = \text{calories} - (\text{protein_g} * 4)$ $\text{fat_g} = \text{remaining} * \text{fat_ratio} / 9$

calculate_carbs(calories DECIMAL, protein_g DECIMAL, fat_g DECIMAL, carbs_ratio DECIMAL)	Calculates carbohydrate grams from remaining calories after protein and fat	remaining = calories - (protein_g * 4) - (fat_g * 9) carbs_g = remaining * carbs_ratio / 4
--	---	---

9. USER OPERATION

9.1 Registration

- Fill in registration form (email, phone, first name, last name, gender, date of birth, height, weight) → click Register
 - Operation: **CREATE | PROCEDURE | TABLE 'User'** — Calls **register_user**(email, phone, first_name, last_name, gender, dob, height, weight) to insert a new user after validating uniqueness of email and phone
 - Operation: **TRIGGER** — t_before_insert_user validates dob is before today
- Click Start Assessment or Make a Booking from success screen

9.2. Assessment

- Enter registered email → click Start Assessment
 - Operation: **CREATE | PROCEDURE | TABLE 'Assessment_Session'** — Calls **start_assessment_session**(email) to create a new session or resume an existing in-progress session, returns session_id
- Select answers for 12 required lifestyle questions + 1 optional question → click Submit Assessment
 - Operation: **CREATE | PROCEDURE | TABLE 'Selects'** — Calls **submit_response**(session_id, question_id, option_text) once per question to insert each response into the Selects table
 - Operation: **TRIGGER** — t_before_insert_selects validates that the option belongs to an active question and enforces single option selection for boolean and single_choice questions
- Click Get My Results

- Operation: **CREATE | PROCEDURE | TABLE 'Assessment_Session'** — Calls **submit_assessment**(session_id, predicted_obesity_level) to finalize the session and compute the personalized nutrition plan
 - Calls **calculate_calories**(weight, height, dob, gender, calorie_adjustment_factor) to compute target daily caloric intake using TDEE formula
 - Calls **calculate_protein**(weight, protein_per_kg) to compute daily protein in grams
 - Calls **calculate_fat**(calories, protein_g, fat_ratio) to compute daily fat in grams
 - Calls **calculate_carbs**(calories, protein_g, fat_g, carbs_ratio) to compute daily carbohydrate in grams
- Operation: **READ | PROCEDURE | TABLE 'User'** — Calls **get_user_by_email**(email) to retrieve user biometric data for ML input, using **calculate_age**(dob) and **calculate_bmi**(height, weight) as derived fields
- Click Book a Consultant → navigates to Booking page
- Click Start New Assessment → clears current session and restarts assessment flow
- Click Save and Exit → session saved as in-progress, user can return via Assessment History

9.3. My Assessments

- Enter registered email → click View History
 - Operation: **READ | QUERY | TABLE 'Assessment_Session'** — queries all assessment sessions for the user ordered by most recent
- For incomplete assessments without saved responses: click Continue → navigates to questionnaire form
- For incomplete assessments with saved responses: click Get Results → runs ML prediction flow above without re-entering the questionnaire
- For any assessment: click Delete → click Yes, Delete to confirm

- Operation: **DELETE | PROCEDURE | Assessment_Session, Selects** — Calls **delete_assessment_session**(session_id, email) to delete the session after validating it belongs to the user; MySQL CASCADE automatically deletes all associated response records in Selects

9.4. Booking

- Enter registered email → click Continue
- Apply filters (date, specialization, consultant name) to narrow down consultant list
 - Operation: **READ | PROCEDURE | TABLE 'Consultant, Obtains, Specialization'** — Calls **search_consultants_by_specialization**(specialization_name) to filter consultants by specialization
 - Operation: **READ | QUERY | TABLE 'Consultant, Obtains, Consultant_Availability'** — Calls **get_consultants_filtered**() to apply combined date, specialization and name filters
- Expand consultant card → select an available slot from the dropdown
 - Operation: **READ | PROCEDURE | TABLE 'Consultant_Availability'** — Calls **get_available_slots**(consultant_email) to retrieve open slots for the selected consultant
- Click Confirm Booking to finalize the reservation
 - Operation: **UPDATE | PROCEDURE | TABLE 'Consultant_Availability'** — Calls **book_slot**(user_email, consultant_email, slot_date, slot_time) to reserve the slot with row-level locking to prevent race conditions
 - Operation: **TRIGGER** — **t_sync_booking_status_on_update** automatically sets **booking_status = 'booked'**

9.5. My Bookings

- Enter registered email → click View My Bookings
 - Operation: **READ | PROCEDURE | TABLE 'Consultant_Availability, Consultant, Obtains'** — Calls **get_booking_history**(user_email) to retrieve all bookings with consultant details and specializations

- Click Update → apply filters → expand consultant card → select new slot → click Confirm Update
 - Operation: **UPDATE | PROCEDURE | TABLE 'Consultant_Availability'** — Calls **update_booking**(user_email, old_consultant_email, old_slot_date, old_slot_time, new_consultant_email, new_slot_date, new_slot_time) to atomically release old slot and reserve new slot within a transaction
 - Operation: **TRIGGER** — t_sync_booking_status_on_update automatically sets old slot to 'open' and new slot to 'booked'
- Click Cancel → click Yes, Cancel to confirm cancellation
 - Operation: **UPDATE | PROCEDURE | TABLE 'Consultant_Availability'** — Calls **cancel_booking**(user_email, consultant_email, slot_date, slot_time) to cancel the booking by clearing the user association from the slot
 - Operation: **TRIGGER** — t_sync_booking_status_on_update automatically resets booking_status = 'open' making the slot available for other users

10. README

10.1 Download & Install

- Python 3.12.7: <https://www.python.org/downloads/>
- MySQL 8.4.0: <https://dev.mysql.com/downloads/mysql/>
- Anaconda (optional): <https://www.anaconda.com/download>

10.2 Setup

1. Clone the code:

Option 1: Using zip file:

```
Unzip the file
```

```
cd livewell
```

Option 2: Cloning the repository:

```
git clone https://github.com/tuyettu1712/livewell-application
```

```
cd livewell-application
```

2. Create environment and install dependencies:

Option 1: With Anaconda:

```
conda create -n livewell_env python=3.12 pip
```

```
conda activate livewell_env
pip install -r requirements.txt
```

Option 2: Without Anaconda:

```
python -m venv livewell_env
source livewell_env/bin/activate           # macOS/Linux
livewell_env\Scripts\activate             # Windows
pip install -r requirements.txt
```

3. Import database

open MySQL Workbench, run

```
SET GLOBAL log_bin_trust_function_creators = 1
```

then import sql/livewell_dump.sql via Server → Data Import.

4. Configure database connection in db/connection.py by your credential:

```
mysql.connector.connect(
    host="localhost",
    user="YOUR_USER",
    password="YOUR_PASSWORD",
    database="livewell")
```

5. Run the application:

```
streamlit run app.py
```

App opens at <http://localhost:8501>

10.3 Dependencies (requirements.txt):

- streamlit==1.56.0,
- mysql-connector-python==9.6.0,
- pandas==3.0.2,
- numpy==2.4.4,
- scikit-learn==1.8.0,
- lightgbm==4.6.0,
- joblib==1.5.3,
- pillow==12.2.0

11. LESSON LEARNED

11.1 Technical expertise gained

We had the practical experience of creating a complete data-driven application with many layers with the LiveWell, such as the user interface, database, and machine learning modules. We enhanced our understanding of MySQL through primary, secondary keys, constraints, programs and abilities as well as user-created functions. We also got to know the impact of schema choices on the application logic particularly the alignment of assessment processes, recommendation rules and booking of consultant services into one system.

On the application front, we enhanced our knowledge on Python-based development on the frontend with Streamlit and Python service modules on the backend side. Another practical experience we obtained was loading and working with a pre-trained machine learning model and applying it to an application workflow. Moreover, we enhanced the debugging skills with the real environment and deployment problems like compatibility with the Python version, MySQL connection, missing procedures, and LightGBM runtime dependencies on macOS.

11. 2. Insights, time management insights, and data domain insights

The key lesson learned during this project was that the implementation process is far more time-consuming than the initial design, particularly when the database, code, and documentation have to remain harmonized. We discovered that, frequently, to change the schema in a single place, it is necessary to update many other areas, such as SQL files, application logic, UML diagrams, and written documentation. This helped us to understand that it is important to test a bit at a time and have all project artifacts in sync rather than delaying updates until the last.

In terms of team work and time management, we got to know how we need to arrange the changes so as to coordinate them. In a number of instances, the local setup or the revised design of one member of the team did not match with that of the other and as a result, this brought confusion in the implementation and documentation. This taught us the importance of talking about design changes in advance, having regular file versions and checking the overall structure before proceeding with application code.

In the domain of health and wellness, we have already discovered the way the data surrounding lifestyle and biometric can be converted into structured obesity-level guidance and nutrition advice. We, too, learned more about the ability of recommendation rules to provide an interpretable interface between machine learning predictions and outputs. This caused the

project to be particularly meaningful, as it combined technical work with one of the areas that directly influence the well-being of people.

11. 3. Realized or contemplated alternative design / approaches to the project

In the development, we had different potential designs of how the assessment part of the system would be but we had to consider and refine them. To start with, we created Assessment_Result and Nutrition_Plan as distinct entities. We later reduced the implementation by integrating these outputs into the very process of the assessment and in the refined conceptual model, we regarded attributes like assessed time, status, outcome of obesity level, and nutrition values as results of an assessment session by a user. This redesign process made us better match the conceptual model with a configuration of the real workflow of the application.

We also thought of other ways of booking consultants. The design that could be used was to establish a separate table of booking or booking-history. But since our application would only have to put the current appointments on the screen and enable the user to amend or cancel one, we chose to use the current-state approach via Consultant-Availability. We also thought we should book history permanently, but we decided that since the scope of the project just required us to maintain merely the existing appointments booked, it was not necessary to book permanent history. These options provided us with more insight into the trade-off between normalization, concept clarity, and implementation.

12. FUTURE WORK

12.1 Planned uses of the database

LiveWell database will accommodate a health and wellness platform that will be capable of extending beyond the present proof of concept. The current database will allow registering users, assessing their own obesity level, giving them their own nutritional advice, looking up and booking an appointment with a consultant, and so on. The database may become the basis of an all-encompassing digital health support system in the future wherein users would overtime

monitor several assessments and compare the change in their health measurements and additionally it may offer more tailored follow-up suggestions.

The database might also facilitate richer interactions with the consultants as in the case of appointment notes, consulting summaries and user-based plans of care. In the long run, the data collected on the assessments and booking information could be stored and analyzed in trends, reporting, and model refinement, which would serve the individual users, along with the service, more effectively.

12.2 Potential areas for added functionality

A complete authentication and system of account management is one of the biggest areas of future work. In the present project, the user identity will be identified by means of the form input, which will suffice only to prove the idea of a project but not to find a platform fit to be used in the production. Another addition could be to have a secure method of login, password control and user, consultant and administration role-based access in the future.

The other noteworthy extension would be to enhance the process of assessment and recommendation. The existing system is based on a pre-trained machine learning model and rule-of-thumb nutrition computing, yet future ones might be able to include retraining the model, visual history of assessment, and more adaptive suggestions given previous user progress. This questionnaire could be further extended with optional questions, more general health indicators, and personalized consultant matching.

In covering the booking part it could cover future features like reminders, retention of appointment history, consultant notes, rescheduling log, and filtering consultants based on other features like ratings or experience. A released system might also scale out of localhost by being able to support cloud hosting, multi-user access, and more robust connecting the frontend, database and ML forward.